

HEIMDALL

SYSTEM ARCHITECTURE & TECHNICAL MANUAL

A Deep Dive into the Concurrency Control, Scaling Design, and Deployment Blueprint of a College In-Out Management Platform.

Target Scale:	15,000+ Registered Students (1,000 active scans/min)
System Core:	Node.js / Express / Redis (BullMQ) / MongoDB Atlas
Security Level:	High Security (JWT + Dynamic TOTP QR + Brute-force Limiting)
Optimizations:	Scan Lock Service, Redis caching, Rate-limiters, Gzip compression
Version:	v2.4.1 (Production Release)

Prepared for Campus Administration & IT Operations Team

1. Executive & Project Overview

The Heimdall Smart Hostel In-Out Management Platform solves a critical logistical bottleneck in large-scale residential universities: managing the exit and entry permissions of thousands of students during rush hours. Traditional paper logbooks create massive delays, are prone to proxy entries, and fail to provide real-time campus occupancy tracking.

Heimdall replaces physical logs with dynamic, time-sensitive encrypted QR codes. The platform integrates a low-latency gate scanner, role-based dashboards, and a background parent-approval messaging queue. Designed with enterprise-grade horizontal scaling practices, Heimdall handles peak student movement with sub-second response times.

Key Deliverables & Performance Objectives:

- %^a Fast Verification** Gate scanning verified in less than 50ms at the API level.
- %^a Concurrency Handling** Support for 1,500–3,000 active concurrent user sessions and up to 1,000 simultaneous scans.
- %^a Zero-Trust Security** Screenshot sharing prevention via cryptographically salted OTP QR structures.
- %^a Resilience & Uptime** Graceful server degradation, connection throttling, and background queuing to run reliably even on free-tier limits.

2. System Stack & Architecture

Heimdall employs a modern decoupled model split into static frontend clients and a stateless, horizontal-scale backend API. Key states are maintained using MongoDB and Redis.

Architecture Breakdown:

- %^a Vite/React Frontend**: Configured as a Single Page Application (SPA). Employs code-splitting, lazy route loading, response caching, and a scanner component built with html5-qrcode. Highly responsive CSS dashboards provide real-time updates via Socket.io.
- %^a Node.js / Express Backend**: stateless API routing layer serving JSON endpoints. Handles JWT verification, input validation, scan authorization, and event streaming.
- %^a Redis Caching & Sync Layer**: Acts as the high-speed data store. Caches Warden dashboard stats to save DB CPU, holds active scan locks, lists active sessions, and drives the BullMQ notification queue.
- %^a MongoDB Atlas Database**: Stores durable records: Student accounts, Warden credentials, detailed check-in/out logs, and Warden-approved Home Visit passes.

Enterprise Technical Stack:

Component	Technology / Package	Strategic Purpose
API Web Server	Express.js + Socket.IO	REST API + Live websocket events
Task Queue	BullMQ + Redis	Non-blocking WhatsApp notifications
Security Headers	Helmet.js + CORS + RateLimit	Protects against XSS, clickjacking, brute-force
Database Mapper	Mongoose (MongoDB 7.0)	Data modeling, indexing, connection pools

3. Database Schemas & Data Model

To achieve optimal performance at scale, the database schemas are highly indexed and structured to minimize expensive multi-collection joins.

Core Schemas Overview:

%^a **1. User Model** (`^User.js`) Stores authentication details, hashed passwords (bcryptjs), and roles: ADMIN, WARDEN, GUARD, STUDENT. Indexed on [email] and [role] for fast access.

%^a **2. In-Out Log Model** (`^InOutLog.js`) Records gate movements. It maps a [studentId], [timestamp], [direction] (in/out), and the [guardId] who executed the scan. Compound indexes on { studentId: 1, timestamp: -1 } ensure instantaneous lookup of current status.

%^a **3. Home Visit Log Model** (`^HomeVisitLog.js`) Tracks extended outpass permits. Links [studentId], [destination], [exitDate], [returnDate], [parentApprovalStatus] (pending, approved, rejected), and [wardenApprovalStatus].

%^a **4. Complaint Model** (`^Complaint.js`) Facilitates hostel grievance tracking. Fields include [studentId], [title], [description], [status] (pending, resolved), and [wardenRemarks].

Critical Database Indexes:

The following MongoDB indexes are created on startup to prevent full-collection scans:

%^a **User Collection** email: 1 (Unique), role: 1

%^a **InOutLog Collection** studentId: 1, timestamp: -1, direction: 1

%^a **HomeVisitLog Collection** studentId: 1, parentApprovalStatus: 1, exitDate: -1

4. Concurrency Control & Security Protocol

Under load, race conditions and duplicate scans present major reliability risks. Two security guards scanning the same student QR code at different gates simultaneously could create corrupted double-logs. Heimdall implements strict concurrency controls to prevent this.

1. Double Scan Locking (`ScanLockService`):

When a QR code is read at a gate, the API immediately requests a distributed lock using Redis with an atomic Lua script:

```
// Redis Lua Atomic NX Lock Execution

const lockKey = `scanlock:${studentId}`;

const acquired = await redis.set(lockKey, "locked", "NX", "EX", 5);

if (!acquired) throw new Error("Duplicate scan detected. Please wait 5 seconds.");
```

2. Dynamic QR TOTP Encryption:

Student QRs do not contain raw student IDs. Instead, they contain a signed, short-lived JWT token payload cryptographically bound to a 60-second time-based window. If a student attempts to screenshot a QR and send it to a friend, the code expires before it can be scanned. This secures the perimeter and prevents unauthorized entry.

5. Telemetry & Load Test Performance

To validate Heimdall under real-world conditions, an automated stress-test suite using Autocannon simulates 3,000 active concurrent users sending up to 1,000 scan requests/minute.

Load Test Setup:

The load testing script (`backend/loadtest/enterprise.js`) issues mock JWTs, spins up parallel connections, and hits key read/write endpoints to check for database connection saturation, memory leaks, and CPU throttling.

Observed Latency Profiles:

- `%^ Readiness Endpoint (/api/ready/)` Average latency: 2ms. (Serves cached checks, zero DB hit).
- `%^ Student Status check (/api/student/status/)` Average latency: 24ms. (Optimized via indexes).
- `%^ Gate Scan Execution (/api/gatescan/scan/)` Average latency: 35ms. (Includes Redis lock verification and DB write).
- `%^ Dashboard Metrics (/api/dashboard/summary/)` Average latency: 5ms. (Served directly from Redis Cache).

Performance Comparison Summary:

Metric Category	Prior to Optimization	Post-Optimization (v2.0)
Max Concurrent Active Sessions	~350 users	1,500+ users
Peak QR Scanning Throughput	~400 scans/min	900+ scans/min
Database CPU Load (at Peak)	100% Saturation	~12% (Cached & Capped)

6. Deployment & Operations Blueprint

Heimdall is designed for simple, zero-downtime deployment on scalable hosting layers.

Hosting Layout:

%^a **Frontend SPA** Deployed on Vercel for instant CDN-edge delivery, Gzip response compression, and low latency.

%^a **Backend API Web Server** Hosted on Render (Standard Tier or higher in production) configured with horizontal auto-scaling based on CPU load.

%^a **Redis Server** Hosted on Upstash or Redis Labs with TLS security enabled (``rediss://``).

%^a **MongoDB Cluster** Hosted on MongoDB Atlas. Capable of scaling from M0 free tier to M30+ cluster based on campus population.

Monorepo Scripts Reference:

%^a ``npm run install:all`` Installs all dependencies across both client and server roots.

%^a ``npm run seed`` Resets and populates the database with standard testing credentials.

%^a ``npm run backend`` / ``npm run frontend`` Starts the backend development server and Vite frontend server concurrently.

%^a ``npm run loadtest`` Triggers standard Autocannon benchmarks to verify system readiness.

End of System Manual.

